Inventory Management System

Java OOP Final Project — Study & Defense Guide

OOP JDBC MySQL Swing GUI

BINUS International University · Business Information Systems

Batch 2029

■ Encapsulation

Q: Show an example of encapsulation in your project?

In Product.java, variables such as productId, name, category, price, and quantity are declared private. They are hidden from external classes and accessed only through public getter and setter methods like getName() and setName().

Q: Why are your variables private?

Making variables private enforces data hiding — it protects the internal state of objects from being directly manipulated or corrupted by external code such as the GUI or utility classes.

Q: Why use getters and setters?

They provide a controlled interface for accessing and modifying private variables. This allows adding validation rules in the future (e.g., ensuring quantity is not negative) without breaking any classes that rely on the object.

■ Inheritance

Q: Where is inheritance used?

ExpiringProduct.java extends the base class Product.java using the keyword 'extends'. This means ExpiringProduct automatically inherits all attributes and behaviours defined in Product.

Q: Why is inheritance useful?

It promotes code reusability and logical hierarchy. ExpiringProduct inherits productId, name, price, etc. from Product automatically and only needs to define the specialised expiryDate attribute and override specific methods.

Q: Could this project work without inheritance?

Yes, but you would need either two separate classes with identical duplicate fields, or a single massive Product class with nullable expiry fields. Both approaches create duplicate code, violate OOP best practices, and are harder to maintain.

■ Polymorphism

Q: Show an example of polymorphism?

In ExpiringProduct.java the getTotalValue() method is overridden to apply a 20% discount:

@Override

public double getTotalValue() {

return getPrice() * getQuantity() * 0.80;

}

The parent Product.java version returns the standard calculation (price * quantity). Java executes the correct version at runtime.

Q: Why is polymorphism beneficial?

It allows processing different product types uniformly. The caller (e.g., the GUI table) does not need if/else checks to determine which class it is dealing with — it simply calls getTotalValue() on a Product reference.

Q: What is method overriding?

Method overriding is when a subclass provides its own implementation of a method already defined in its parent class, keeping the exact same method signature (name, parameters, and return type).

■ Abstraction

Q: What is abstraction?

Abstraction hides complex implementation details and exposes only essential features of a system. It separates what a system does from how it does it.

Q: Did you use interfaces or abstract classes?

Yes — the project uses the Searchable.java interface, which defines contracts for searchById() and searchByName(). InventoryManager implements these methods.

Q: Why use an interface?

Defining Searchable decouples the search structure from concrete implementation details, making the code modular and easier to extend or test independently.

■ Code-Based Questions

Q: What is the most important class?

InventoryManager.java is the primary logic controller — it coordinates in-memory collections (ArrayList, HashMap) and connects the GUI to the database layer. Product.java is the core data model.

Q: Which class communicates with MySQL?

DatabaseManager.java is solely responsible for all database interactions — connecting to MySQL, executing prepared statements, and mapping result sets to Java objects.

Q: Which class controls the GUI?

The application has multiple views: LoginScreen.java acts as the home menu, directing users to ManagerGUI.java, CashierGUI.java, or CheckPriceGUI.java depending on role.

Q: Why did you use linear search in searchByName()?

Product names are unsorted and not unique. Linear search allows finding partial matches (e.g., searching 'milk' matches 'Fresh Milk') via simple iteration. No binary search is possible without a sorted, unique key.

Q: Why did you create ProductComparator?

To separate custom comparison logic (sort alphabetically by name or numerically by price) from the Product data model itself. This follows the Single Responsibility Principle.

■■ JDBC

Q: What is JDBC?

JDBC (Java Database Connectivity) is a standard Java API that allows Java applications to connect to relational databases and execute SQL statements.

Q: How is the connection established?

By loading the driver and calling DriverManager.getConnection(DB_URL, DB_USER, DB_PASS) inside DatabaseManager.java.

Q: What is a PreparedStatement and why use it over Statement?

A PreparedStatement is a precompiled SQL statement that uses placeholders (?) for parameters. Regular Statement compiles SQL dynamically on every call and is vulnerable to SQL injection. PreparedStatement treats user input strictly as literal values — never executable code.

Q: What is SQL injection?

An attack technique where malicious SQL commands are embedded into text fields to alter the database query structure, potentially allowing attackers to access, modify, or delete private data.

Q: What is a ResultSet?

A ResultSet is a table of data representing a query result. It maintains a cursor pointing to the current row, which you traverse using rs.next().

■■ Error Handling

Q: What happens if MySQL is offline?

A SQLException is thrown. DatabaseManager.java catches it, logs the error, and the GUI shows a friendly JOptionPane popup. The program remains active and does not crash.

Q: What happens if invalid data is entered?

The system validates inputs before saving. If non-numeric values are entered into numeric fields (price/quantity), a NumberFormatException is caught and triggers an 'Invalid numeric input' warning dialog.

Q: How do you handle exceptions?

Using standard try-catch blocks and Java's Try-With-Resources structure to automatically close database objects (Connection, PreparedStatement, ResultSet) and prevent resource leaks.

Q: Why is exception handling important?

It guarantees application stability, prevents unexpected crashes, ensures connection resources are cleaned up, and guides the user when an action fails.

■■ Design Questions

Q: Explain your class diagram.

The system uses a model-controller-view architecture:

- Product & ExpiringProduct → core data models
- InventoryManager → data/business logic controller
- DatabaseManager → database connection & persistence
- GUI classes (ManagerGUI, CashierGUI, etc.) → view layer

Q: Explain the ERD.

Single table named 'products' with columns: product_id (VARCHAR PK), name (VARCHAR), category (VARCHAR), price (DOUBLE), quantity (INT), is_perishable (INT), expiry_date (VARCHAR).

Q: Why did you separate GUI and database code?

To follow Separation of Concerns. The database layer is independent of the GUI, allowing you to test database methods separately or swap databases without touching UI code.

Q: What software engineering principles did you apply?

- SRP (Single Responsibility Principle): each class has one distinct job.
- DRY (Don't Repeat Yourself): inheritance avoids duplicating Product fields.
- Program to an interface: InventoryManager implements Searchable, not a concrete class.

■ Critical Thinking

Q: If the database contains 1 million records, what problems occur?

Loading all records into a local ArrayList and HashMap would cause memory depletion and slow down sorting/searching. Rendering 1M rows in a JTable would also freeze the UI thread.

Q: How would you improve performance at scale?

- Add database indexes on frequently searched columns.
- Implement pagination using SQL LIMIT and OFFSET (e.g., 50 rows at a time).
- Perform sorting and filtering directly in SQL instead of loading everything into Java memory.

Q: How would you add user authentication?

Create a 'users' table (username, password_hash, role). Build a login screen that verifies credentials against the table before granting access, hashing passwords with bcrypt or PBKDF2 + salt.

Q: How would you migrate this to a web-based system?

Backend: re-purpose business logic as a RESTful API (Spring Boot or Node.js).

Frontend: rebuild UI in React, Vue, or Angular.

Database: keep MySQL, deploy to a cloud provider.

Q: What would you improve if given another month?

- Add automated unit tests with JUnit.
- Implement role-based permissions (manager vs. cashier).
- Use HikariCP database connection pooling for optimised connection reuse.

■ Deep Dive: mapRowToProduct()

```
private Product mapRowToProduct(ResultSet rs) throws SQLException {
boolean isPerishable = rs.getInt("is_perishable") == 1;
if (isPerishable) {
return new ExpiringProduct(
rs.getString("product_id"),
rs.getString("name"),
rs.getString("category"),
rs.getDouble("price"),
rs.getInt("quantity"),
rs.getString("expiry_date")
);
} else {
return new Product(
rs.getString("product_id"),
rs.getString("name"), ...);
}
}
```

Line 1	Private helper method — maps one SQL row to a Java object.
Line 2	Reads is_perishable column; value 1 = perishable.
Line 3-11	Constructs an ExpiringProduct if perishable, passing expiry_date.
Line 12-15	Otherwise constructs a standard Product object.